# MiniPar-Improvement — Relatório de Projeto AB2 · Compiladores · UFAL

> **Versão:** 2026.2 (draft em Markdown para revisão, antes da geração do PDF) **Atualizado a partir de:** `RelatorioMinipar2026-1-Compilador-GeracaoCodigoExe-10,0-okokok.pdf` (15/06/2026) **Data desta versão:** novembro de 2026 **Disciplina:** Compiladores — Prof. Arturo Hernandez Dominguez **Instituição:** Universidade Federal de Alagoas — Instituto de Computação — Ciência da Computação **Repositório (2026.2):** https://github.com/johnUFAL/minipar-improvement-2026.2 **Repositório (2026.1, base):** https://github.com/luizwhirl/minipar-improvement

## Equipe

**Turma 2026.2 (esta versão):**

- João Victor Duarte do Nascimento — johnUFAL
- Rian Antonio da Silva Gaião — ryanthecomic
- José Guilherme de Almeida Novais — GuilhermeANovais
- Daniel de Araujo Albuquerque — Daniel-de-Araujo91
- Guilherme Alessander da Silva Fernandes — Alessander-Magno

**Turma 2026.1 (base da versão anterior):**

- Emanuele Vitória de Jesus Lima
- José Lucas de Oliveira Quintela
- Luiz Miguel de Melo Bomfim

---

# Sumário

1. Introdução
    - 1.1 Arquitetura geral
2. A linguagem MiniPar 2026.2
    - 2.1 O que mudou em relação a 2026.1
    - 2.2 BNF resumida implementada (EBNF)
3. Analisador léxico
4. Analisador sintático (parser + AST)
5. Analisador semântico e tratamento de erros
6. Tabela de símbolos
7. Geração de código intermediário TAC
8. Geração de código C++ e compilação com GCC
9. Interface web (IDE didática)
    - 9.1 Hub de Conexão e execução em dois computadores
10. Product Backlog e Sprint Backlogs
    - 10.1 Modelagem UML
11. Testes de integração e programas de exemplo (T1–T11)
12. Implementação
    - 12.1 Tecnologias utilizadas
    - 12.2 Fluxo de execução
    - 12.3 Observações técnicas e limitações
13. Conclusão
14. Links úteis
15. Referências

# 1. Introdução

Este relatório descreve tecnicamente o projeto **minipar-improvement 2026.2**, um compilador educacional para a linguagem **MiniPar**. Diferentemente de um interpretador, o projeto implementa uma cadeia de compilação em quatro fases:

> **fonte `.mp` → análise léxica e sintática (AST) → análise semântica e tabela de símbolos → geração de TAC (código de três endereços) → transpilação para C++17 → compilação com `g++` → binário executável**

O objetivo é atender ao enunciado 2026.2: blocos sequenciais e paralelos (`SEQ`/`PAR`), orientação a objetos (classes, herança, `new`, `self`), funções com padrões e retorno tipado, estruturas de controle, paralelismo por núcleo (`fork ... em core(N)`) e por máquina (`fork ... em host("...")`), canais de comunicação TCP tipados, entrada/saída, comentários iniciados por `#`, interface CLI e IDE web didática, testes de software e documentação alinhada à gramática formal.

A versão 2026.2 é uma **revisão profunda** da base 2026.1, não apenas uma reentrega. A mudança mais visível é a adoção de sintaxe por **indentação estilo Python** (`INDENT`/`DEDENT`, 4 espaços por nível), substituindo chaves `{ }` e `;`. Canais deixam de ser expressão `c_channel(ip, porta)` e passam a ser **declaração lógica no topo do programa** (`c_channel nome computador_1 computador_2`), com configuração física via `minipar.hosts`. O backend passa de C++20 com `-O3` fixo para **C++17 com `-O2` por padrão** e `-O0`–`-O3` configurável. Ver seção 2.1 para a tabela completa de mudanças.

## 1.1 Arquitetura geral

| Componente | Arquivo | Responsabilidade |
|---|---|---|
| Entrada CLI | `main.py`, `interfaces/command_line.py` | `python main.py` (menu interativo) ou comando `minipar` instalado via `uv sync`; subcomandos `compilar`, `tokens`, `ast`, `tac`, `cpp`, `build`, `run`; inspeção de fases com `--tokens/--ast/--tac/--emit-cpp`; códigos de saída 1 (uso/arquivo) e 2–7 por fase (léxica, sintática, semântica, geração, build, execução). |
| Menu terminal | `menu.py`, `interfaces/cli.py`, `interfaces/test_orchestrator.py` | Menu interativo em terminal (título "Menu de Testes - Compilador MiniPar 2026.2"; opções: rodar todos os testes, escolher um teste, Conexão/Hub); monitora sessões multi-terminal (participante/monitor de progresso). Não inicia processos multi-terminal por si — ver `docs/rede.md` e `scripts/run_multiterminal.py`. |
| IDE Web | `app.py`, `interfaces/web/api.py`, `interfaces/web/frontend/` | Servidor Flask, editor CodeMirror 5.65.13 servido **localmente** (sem CDN), abas de tokens/AST/símbolos/TAC/C++/execução, terminal, histórico de compilação, painel de benchmark, **download do executável** (`.exe`) e página de **Ajuda**. |
| Hub de Conexão | `interfaces/conexao/` (`hub.py`, `cliente.py`, `protocolo.py`, `execucao.py`, `terminal.py`), | Hub TCP com salas (lobby público + salas privadas de 2 pessoas), chat, convites de execução e coordenação da |

| Componente | Arquivo | Responsabilidade |
|---|---|---|
| | `interfaces/web/conexao_api.py`, `interfaces/web/conexao_execucao.py` | execução de programas cliente/servidor em dois computadores, pelo editor. Ver §9.1. |
| Contratos | `components/interfaces.py` | 7 interfaces ABC: `ILexico`, `ISintatico`, `ISemantico`, `ITabelaSimbolos`, `IGeradorTAC`, `IGeradorCpp`, `IControle`. |
| Controle | `components/controle/c_controle.py`, `components/compilador.py` | `CControle` (pipeline injetável, testável com fakes) e função `compile_file(...)` (fluxo concreto CLI que instancia componentes). `CompiladorMiniPar` apenas delega a `IControle` e lê o arquivo-fonte. |
| Léxico | `components/lexico/c_lexico.py`, `components/lexico/token.py` | Autômato escrito à mão, consumo sob demanda via `next_token()`/`peek()`, pilha de indentação, buffer de tokens. |
| Sintático | `components/sintatico/c_sintatico.py`, `components/sintatico/ast.py`, `components/sintatico/visitor.py` | Parser descendente recursivo LL(1) que produz AST em dataclasses frozen + Visitor/ASTPrinter. |
| Semântico | `components/semantico/c_semantico.py` | Duas passagens: coleta de declarações e validação com inferência de tipos, herança, override, concorrência e canais. |
| Tabela de símbolos | `components/tabela_simbolos/c_tabela_simbolos.py` | Árvore de escopos com pilha de ativos, exportação serializável e busca de membros com herança. |
| Diagnósticos | `erros/diagnosticos.py` | `Diagnostic` imutável + `DiagnosticBag` com formatação origem→código, aliases PT (`Fase`, `Severidade`, `ColetorDiagnosticos`) e saída JSON. |
| TAC | `components/tac/c_gerador_tac.py`, `components/tac/executor_tac.py`, `components/tac/otimizador_tac.py`, `components/tac/instrucoes.py` | Gera texto TAC `.tac` de três endereços, interpreta TAC em memória e aplica otimizações simples (visão didática na IDE; o binário C++ não muda por causa do toggle). |
| Codegen + GCC | `components/codegen/c_gerador_cpp.py`, `components/codegen/driver_gcc.py` | Transpila TAC → C++17 (`.cpp`) e compila/linkifica com `g++`, incluindo runtime. Mapeia diagnósticos do C++ gerado de volta para linhas MiniPar via `// MiniPar line N`. |
| Runtime C++ | `runtime/minipar_rt.hpp`, `runtime/canal.cpp`, `runtime/paralelo.cpp`, `runtime/worker.cpp` | `MiniParChannel` (TCP tipado), `minipar_parallel` (threads + afinidade `core`), helpers de builtins, matrizes, E/S e worker de `fork` remoto. Compilado junto com cada programa. |

**Estrutura do repositório:**

```
components/
  interfaces.py
  lexico/          c_lexico.py, token.py
  sintatico/       c_sintatico.py, ast.py, visitor.py
  semantico/       c_semantico.py
  tabela_simbolos/ c_tabela_simbolos.py
  tac/             c_gerador_tac.py, executor_tac.py, otimizador_tac.py, instrucoes.py
  codegen/         c_gerador_cpp.py, driver_gcc.py
  controle/        c_controle.py
erros/             diagnosticos.py
runtime/           minipar_rt.hpp, canal.cpp, paralelo.cpp, worker.cpp
interfaces/        cli.py, command_line.py, web/api.py, web/benchmark.py,
web/conexao_api.py,
                   web/conexao_execucao.py, web/frontend/, conexao/ (hub, cliente,
protocolo)
programas_teste/   programas MiniPar (.mp, incluindo t01_multiterminal/ e
dois_computadores/)
benchmarks/        baselines Python + benchmark_fork_core.py
docs/              gramatica.md, backlog.md, rede.md, uml/, relatorio/
tests/             unitarios/, integracao/, programas/, runtime/
scripts/           run_testes_2026_2.py, run_multiterminal.py, benchmark_t08_t10.py,
benchmark_t04.py
```

---

## 2. A linguagem MiniPar 2026.2

A linguagem combina sintaxe inspirada em Python (indentação, `if/elif/else`, `for x in ...`, `def`, `class`) com blocos explícitos `seq` e `par`, concorrência nomeada (`fork`/`join`) e comunicação por canais lógicos.

- **`seq`/`par`** (aceitos também em maiúsculas, `SEQ`/`PAR`) delimitam blocos sequenciais e paralelos. O bloco aceita suíte indentada após `:` ou bloco entre `{ }`; todos os programas de aceitação usam `seq {`/`par {`. Cada filho direto de `par` vira uma thread; para agrupar várias instruções numa thread usa-se `seq` aninhado.
- **Canais:** `c_channel nome computador_1 computador_2` no topo. `nome.send(...)` / `nome.receive(...)` trocam mensagens tipadas. Os nomes são lógicos; endereço/porta ficam em `minipar.hosts` (ver `docs/rede.md`), sem recompilar.
- **Tipos de valor:** `bool`, `int`, `float`, `string`, `char`, `list`, mais classes do usuário. `float` e `list` entram em 2026.2 para cobrir T3–T7. `char` é `std::string` de 1 caractere no C++. `void` só em retorno.
- **Integração com E/S:** `print(...)` e `input()`.
- **Builtins (nomes pré-definidos, não sintaxe):** `print`, `input`, `len`, `range`, `str`, `int`, `float`, `exp`, `random`, `random_uniforme(min,max)`, `inf`, `clock`, `agora`, `format_float(x,casas)`, `is_number(s)`, `max`, `sum`, `parse_ints(s)`, `matrix(linhas,colunas,valor)`, mais string/arquivo: `char_at`, `substring`, `is_digit`, `is_alpha`, `is_space`, `write_file`. Métodos: lista `append`/`pop`, string `split`/`strip`, canal `send`/`receive`.

| Recurso | Situação em 2026.2 |
|---|---|
| Tipos/literais | `bool`, `int`, `float`, `string`, `char`, `list` + classes. `number`/`dict` removidos. |
| Controle de fluxo | `if/elif/else`, `while`, `do … while`, `for x in expr`, `break`, `continue`, `return`. `for` estilo C removido. |
| OO | `class [extends]`, `new`, `self` (alias `this`), atributos declarados no corpo com valor inicial, construtor `init`, métodos `def/func`. |
| Funções | `def` ou `func`, params com tipo opcional e padrão literal, retorno `-> tipo` opcional, recursão. |

| Recurso | Situação em 2026.2 |
|---|---|
| Biblioteca nativa | Acima, plus `clock/agora`, `format_float`, `is_number`, `parse_ints`, `matrix` e builtins de string/arquivo. |
| Coleções | Listas literais `[...]`, indexação, atribuição por índice, `matrix(...)` como builtin (listas de listas). |
| Concorrência | `PAR`, `fork t em core(N):`, `fork t em host("pc2"):`, `join t1, t2`. |

### 2.1 O que mudou em relação a 2026.1

| # | Decisão 2026.2 (ver `docs/gramatica.md` §1) | Impacto |
|---|---|---|
| 1 | Blocos por indentação `INDENT`/`DEDENT` (4 espaços) | Léxico com pilha de indentação; tab é erro. |
| 2 | `float` e `list` nativos; `c_channel` vira entidade nomeada, não tipo | Inferência e C++ tipados; canais só via `decl_canal`. |
| 3 | Sem comprehension, tuplas, `zip`/`enumerate`, f-strings, `try/except`, ternário | Reescritas com `while`/`for`/laços; parser segue LL(1). |
| 4 | Precedência `or < and < not < comparações < + - < * / % < - unário < postfixo` | Um não-terminal por nível; AST já com precedência certa para TAC. |
| 5 | `print`/`input` reservados; `sys.stdin.readline()` não vira alias | `input()` cobre o caso dos testes. |
| 6 | `fork t em core(n):` / `fork t em host("h"):` sintáticos; `join` aceita lista | Alocação vira sintaxe; runtime respeita CPU/máquina. |
| 7–8 | `new` sempre para instanciar; atributos no corpo da classe; `init` como construtor; `self` | C++ conhece membros antes de emitir métodos. |
| 9–10 | `+=`/`-=`/`*=`/`/=` açúcar; params tipados/opcionais com padrão; `-> tipo` | Cobre T3–T5; preenche `tipos_var`. |
| 11–14 | Quebra de linha ignorada em `(`/`[`; `seq`/`par` aceitam suíte com `:` ou bloco `{ }`; filho de `par` = thread; um bloco raiz | Listas multilinha, comentários não quebram indentação. |
| 15 | Canal só no topo; `c_channel` não é tipo | Remove `c_channel(ip,porta)` como expressão. |
| 16–18 | Sem comparação encadeada; `void` só em retorno; sem blocos vazios/`pass` | Erro sintático/semântico explícito em vez de ambiguidade. |
| 19 | Removidos: `for` C, `dict`, `number`, `c_channel(ip,porta)` | `var` mantido como açúcar; `matrix` volta como builtin. |
| 20 | `do … while` **reintroduzido** (exigido pelo enunciado): `do` suite `while` expr | Palavra-chave `do`; o nó `DoWhileStmt` já existia na AST/semântico/TAC e agora é produzido pelo parser; `break`/`continue` valem dentro do laço. |

Aliases de compatibilidade mantidos: `func`/`def`, `this`/`self`, `SEQ`/`seq`, `PAR`/`par`, `{ }`/`;`, `&&`/`||` normalizados para `and`/`or`. O arquivo `docs/uml/BNF-grammaMiniPAR2026-1.md` preserva a BNF antiga para comparação.

### 2.2 BNF resumida implementada (EBNF)

Referência completa: `docs/gramatica.md` §3 (EBNF LL(1)). Notação: `{x}` zero ou mais, `[x]` opcional, MAIÚSCULAS = token.

```
(* Programa e canais *)
programa       = { decl_canal | decl_funcao | decl_classe } bloco_raiz EOF ;
bloco_raiz     = bloco_seq | bloco_par ;
decl_canal     = "c_channel" IDENT IDENT IDENT NEWLINE ;

(* Funções e classes *)
decl_funcao    = ("def"|"func") IDENT "(" [params] ")" ["->" tipo_retorno] suite ;
params         = param {"," param} ;
param          = IDENT [":" tipo] ["=" literal] ;
tipo           = "bool"|"int"|"float"|"string"|"char"|"list"|IDENT ;
tipo_retorno   = tipo | "void" ;
decl_classe    = "class" IDENT ["extends" IDENT] suite_classe ;
suite_classe   = ":" NEWLINE INDENT membro {membro} DEDENT | "{" {membro} "}" ;
membro         = decl_atributo | decl_metodo ;
decl_atributo  = ["var"] IDENT [":" tipo] "=" expr fim_stmt ;
decl_metodo    = decl_funcao ;                (* init = construtor *)

(* Blocos e concorrência *)
suite          = ":" NEWLINE INDENT stmt {stmt} DEDENT | "{" {stmt} "}" ;
fim_stmt       = NEWLINE | ";" ;
bloco_seq      = ("SEQ"|"seq") suite ;
bloco_par      = ("PAR"|"par") suite ;
fork_stmt      = "fork" IDENT ["em" alocacao] suite ;
alocacao       = "core" "(" expr ")" | "host" "(" (STRING|IDENT) ")" ;
join_stmt      = "join" IDENT {"," IDENT} fim_stmt ;

(* Comandos *)
stmt           = bloco_seq | bloco_par | fork_stmt | join_stmt
               | if_stmt | while_stmt | do_while_stmt | for_stmt
               | return_stmt | break_stmt | continue_stmt | simples_stmt ;
if_stmt        = "if" expr suite {"elif" expr suite} ["else" suite] ;
while_stmt     = "while" expr suite ;
do_while_stmt  = "do" suite "while" expr fim_stmt ;
for_stmt       = "for" IDENT "in" expr suite ;
return_stmt    = "return" [expr] fim_stmt ;
break_stmt     = "break" fim_stmt ;
continue_stmt  = "continue" fim_stmt ;
simples_stmt   = ["var"] IDENT [":" tipo] "=" expr fim_stmt
               | expr [":" tipo "=" expr | op_atrib expr] fim_stmt ;
op_atrib       = "=" | "+=" | "-=" | "*=" | "/=" ;

(* Expressões — menor para maior precedência *)
expr           = expr_or ;
expr_or        = expr_and {("or"|"||") expr_and} ;
expr_and       = expr_not {("and"|"&&") expr_not} ;
expr_not       = "not" expr_not | expr_comp ;
expr_comp      = expr_arit [op_rel expr_arit] ;  (* sem encadeamento *)
op_rel         = "=="|"!="|"<"|">"|"<="|">=" ;
expr_arit      = expr_termo {("+"|"-") expr_termo} ;
expr_termo     = expr_unario {("*"|"/"|"%") expr_unario} ;
expr_unario    = "-" expr_unario | expr_posfixo ;
expr_posfixo   = primario {"." IDENT ["(" [args] ")"] | "[" expr "]"} ;
primario       = INT | FLOAT | STRING | CHAR | "true"|"false" | ("self"|"this")
               | IDENT ["(" [args] ")"] | "new" IDENT "(" [args] ")"
               | lista | "(" expr ")" ;
lista          = "[" [args] "]" ;
args           = expr {"," expr} ;
literal        = ["-"] (INT|FLOAT) | STRING | CHAR | "true"|"false" ;
```

Chamada de método, envio e recebimento são `expr_posfixo` como instrução: `neuronio.treinar()`, `calculadora.send(op,v1,v2)`, `calculadora.receive(op,v1,v2)` (argumentos preenchidos) ou `x = calculadora.receive()`.

**Verificação LL(1):** todos os pontos de decisão têm FIRST disjuntos, com uma exceção documentada — `simples_stmt` lê primeiro a `expr` e só depois decide se era chamada ou atribuição, com validação no semântico. `INDENT`/`DEDENT` são produzidos pelo léxico comparando indentação com pilha.

**Cobertura de testes (gramática → T1–T11):** T1 `decl_canal`+`send/receive`; T2 `PAR` com dois `SEQ`, `while`, `*=`; T3 `class`+`init`+padrão, `new`, `self.peso +=`; T4 `-> float`, `exp`, listas aninhadas, `for i in range(len(x))`; T5 herança/listas/`new`/multilinha; T6 método recursivo, `input/split/is_number`; T7 `def`+`continue`; `do … while` coberto por testes dedicados (parser, TAC e execução, com `break`/`continue`); T8–T10 `fork ... em core(N)` + `join`; T11 builtins de string (`char_at`, `substring`, `is_digit`, `is_alpha`, `is_space`, `write_file`) + classe com lexer/parser/gerador dentro do programa. Reescritas sem comprehension/tupla/`zip`/bitwise/`try`/`turtle` estão em `docs/gramatica.md` §5.

**Regras semânticas de tipos (resumo):** coleta de funções/classes antes de validar corpos (recursão e chamadas futuras); `int`→`float` implícito, `float`→`int` só explícito; `/` sempre `float`; primeira atribuição declara no escopo atual, seguintes respeitam tipo; condições booleanas; aridade/tipos de chamadas verificados; **cada ramo de `PAR` e corpo de `fork` não pode escrever em variável externa** (escopo próprio); `send`/`receive` exigem canal declarado e assinaturas compatíveis (`receive()` = 1×`string`); cada `join` casa com `fork` anterior no mesmo bloco e vice-versa.

---

## 3. Analisador léxico

**Implementação:** `components/lexico/c_lexico.py` (`Lexer.next_token`, `_scan_step`, `_scan_indentation`, `_apply_indentation`, `_scan_token`, `_scan_newline`, `_finish`), tokens em `components/lexico/token.py`.

O léxico é um autômato escrito à mão, com consumo sob demanda (`next_token()`/`peek()`) e buffer. Pontos-chave em 2026.2:

- Remove BOM inicial; `INDENT_WIDTH = 4` espaços por nível; **tabulação na indentação é erro léxico** (evita ambiguidade); largura inconsistente é diagnosticada sem interromper a tokenização.
- Pilha de indentação `[0]`; ao início de linha (fora de delimitadores abertos), compara largura com topo: maior → `INDENT`, menor → `DEDENT`(s) até alcançar nível, sem correspondência → erro + recuperação.
- Linhas vazias/só `#` não geram `NEWLINE`; `NEWLINE` dentro de `(…)`/`[…]` é ignorado (listas multilinha — Teste 5). Comentário `#` até fim de linha é descartado em qualquer posição.
- Pilha de delimitadores valida `(`/`[`/`{` sem fechar; ao EOF emite `NEWLINE` final se preciso, um `DEDENT` por nível aberto, registra delimitadores não fechados e `END_OF_FILE`.
- `KEYWORDS` inclui `SEQ`/`PAR` (maiúsculos) mais aliases de migração: `this`→`SELF`, `func`→`DEF`, `&&`→`AND`, `||`→`OR` (em mapas de operadores); dicas para quem vem de 2026.1 (`!`→`use 'not' ou '!='`, `&`→`use 'and'`, `|`→`use 'or'`).
- Literais: `INT`, `FLOAT` (separados em 2026.2), `STRING` (`"..."` com `\n \t \" \\`), `CHAR` (`'...'` com `\n \t \' \\`), `IDENT` ASCII (_/letra ASCII, sem acento — acentos só em strings/comentários), dígitos ASCII `0-9` (evita `str.isdigit()` Unicode).
- Operadores de 1–2 chars: aritméticos, comparação, `= += -= *= /=`, `->`; delimitadores `( ) [ ] . , : { }`.

```
PSEUDOCÓDIGO DO ANALISADOR LÉXICO (resumo fiel a c_lexico.py)

inicializar(fonte):
    remover BOM; cursor = início; linha = 1; coluna = 1
    pilha_indentação = [0]; delimitadores_abertos = []
    marcar início de linha; criar buffer de tokens

próximo_token():
    enquanto buffer não contém token:
        se início de linha e nenhum delimitador aberto:
```

```
            consumir espaços/CR/tabulações; erro se tab na indentação
            se linha vazia ou comentário '#': consumir até newline; continuar
            comparar largura com topo:
                maior → empilhar, emitir INDENT
                menor → desempilhar, emitir DEDENT até alcançar largura
                sem nível correspondente → erro + ajustar para recuperação
        reconhecer próximo token: ignorar espaços e comentários '#'
            palavra/palavra-chave, número, string ou caractere
            operador de 1 ou 2 caracteres; validar pilha de delimitadores
            emitir token com lexema, valor e localização
        ao newline fora de parênteses/colchetes:
            emitir NEWLINE se a linha lógica continha tokens
    ao fim da fonte:
        emitir NEWLINE final se necessário
        emitir um DEDENT por nível aberto; registrar delimitadores não fechados
        emitir END_OF_FILE
    devolver primeiro token do buffer
```

Catalogação de tokens principais:

| Categoria | Tokens principais |
|---|---|
| Estrutura | `NEWLINE`, `INDENT`, `DEDENT`, `END_OF_FILE` |
| Palavras reservadas | `SEQ PAR fork join em core host c_channel class extends new self this def func var return print input if elif else while do for in break continue and or not true false bool int float string char list void` |
| Literais/ident | `INT_LITERAL`, `FLOAT_LITERAL`, `STRING_LITERAL`, `CHAR_LITERAL`, `BOOL_LITERAL`, `IDENTIFIER` |
| Operadores | `+ - * / %`, `== != < > <= >=`, `= += -= *= /=`, `->`, `&& |
| Delimitadores | `( ) [ ] . , : { }` |
| Inválido | `UNKNOWN` (com dica de migração 2026.1 quando aplicável) |

## 4. Analisador sintático (parser + AST)

**Implementação:** `components/sintatico/c_sintatico.py` (`Parser.parse`, `_suite`, `_statement_recovering`, `_synchronize`, `_statement`, `_simple_statement`, `_expression` → `_parse_or` … `_primary`, `_match_block_keyword` para `seq`/`par` minúsculos), AST em `components/sintatico/ast.py`, navegação em `components/sintatico/visitor.py` (`ASTVisitor`, `ASTPrinter`, `para_dict`).

O parser é **descendente recursivo**. Cada produção de §2.2 tem um método. A precedência é a cadeia de chamadas: `or → and → not → comparação → soma → produto → unário → posfixo/primário`. Recursão à esquerda eliminada com `{…}` e associatividade à esquerda na AST. Comparação encadeada (`a < b < c`) é **erro sintático**, não `(a<b)<c`.

```
PSEUDOCÓDIGO DO ANALISADOR SINTÁTICO (resumo fiel a c_sintatico.py)

parse():
    ignorar NEWLINEs iniciais
    enquanto não chegou ao bloco raiz:
        analisar declaração de canal, função ou classe
        se erro: avançar até NEWLINE, DEDENT ou EOF e retomar
    analisar bloco raiz seq/par; exigir EOF
    construir Program(statements) com declarações e bloco raiz
```

```
analisar_bloco():
    aceitar bloco entre chaves ou ':' NEWLINE INDENT ... DEDENT
    enquanto não chegou ao terminador do bloco:
        analisar uma instrução
        se erro: sincronizar até fim da instrução/limite do bloco
    rejeitar bloco vazio

analisar_expressão():
    or chama and; and chama not; not chama comparação
    comparação chama soma e aceita no máximo um operador comparativo
    soma analisa produto repetido por '+'/'-'; produto por '*','/','mod'
    unário consome '+'/'-' recursivamente ou chama posfixo
    posfixo estende primário com acesso a membro, método ou índice
    primário reconhece literais, identificadores, chamadas, listas, input, new, (...)
    construir nós AST à medida que cada produção é reconhecida
```

Recuperação é **por instrução** (sincroniza em `NEWLINE`/`DEDENT`/`EOF`), respeitando `}`/`DEDENT` para não engolir o resto de um bloco válido. O `fim_stmt` aceita `NEWLINE` ou `;`; `suite` aceita `:` indentado ou `{ }`.

**AST:** todas frozen dataclasses com base `ASTNode(line, column, tipo_inferido)` e `definir_tipo_inferido()` / `para_dict()`. Principais nós em 2026.2:

| Grupo | Nós |
|---|---|
| Declarações/raiz | `Program(statements)`, `FuncDecl(name, params, body, return_type)`, `MethodDecl(FuncDecl)`, `Parameter(name, type, default_value)`, `ClassDecl(name, superclass, attributes, methods)`, `VarDecl(name, type, init_value)`, `AttributeDecl(VarDecl)`, `ChannelDecl(name, first_host, second_host)` |
| Comandos | `SeqBlock`, `ParBlock`, `Assignment`, `PropertyAssign`, `IndexAssign`, `PrintStmt`, `SendStmt`, `ReturnStmt`, `BreakStmt`, `ContinueStmt` |
| Fluxo | `IfStmt(condition, then, else)` (com `elif` aninhado), `WhileStmt`, `DoWhileStmt(body, condition)` (produzido pelo parser a partir de `do … while`; no TAC usa os rótulos `DO_n`, `DO_CONDITION_n`, `ENDDO_n`), `ForStmt(iterator_name, iterable, body)` |
| Concorrência | `ForkStmt(task_name, target_kind, target, body)`, `JoinStmt(task_names)` |
| Expressões | `NumberExpr`, `StringExpr`, `CharExpr`, `BoolExpr`, `IdentifierExpr`, `BinaryExpr(op,left,right)`, `UnaryExpr`, `ListLiteral`, `MatrixCreateExpr`, `FuncCallExpr`, `MethodCall`, `PropertyAccess`, `NewExpr`, `ThisExpr`, `IndexExpr`, `InputExpr`, `CChannelExpr`/`ReceiveExpr` (legado) |

> **Nota de modelagem:** `tac_planejado.puml` propõe classes `InstrucaoTAC`/`TACAtribuicao`/etc. como modelo futuro — hoje o TAC é texto interpretado linha a linha, não objetos tipados.

---

## 5. Analisador semântico e tratamento de erros

**Implementação:** `components/semantico/c_semantico.py` (`SemanticAnalyzer.analyze`, `_collect_declarations`, `_check_inheritance_cycles`, `_check_method_overrides`, `validate_node`, `_infer_type`, `_check_concurrent_write`, `_validate_channel_signatures`), diagnósticos em `erros/diagnosticos.py`.

A análise faz **duas passagens**:

1. **Coletar declarações:** registrar canais, funções e classes no escopo global; registrar atributos/métodos em escopos de classe; verificar ciclos de herança e compatibilidade de sobrescritas. Permite recursão e chamadas a

funções/métodos declarados depois.
2. **Validar em ordem:** entrar no escopo de bloco/função/classe/laço/ramo concorrente; inferir tipos de expressões (armazenado em `tipo_inferido` do nó); validar declarações/atribuições, chamadas (aridade/tipos), condições booleanas, retornos (todo caminho), membros herdados e índices; marcar escopos concorrentes e verificar escritas externas; registrar `fork`/canais para validar `join` e assinaturas `send`/`receive`.

```
PSEUDOCÓDIGO DO ANALISADOR SEMÂNTICO (resumo fiel a c_semantico.py)

analyze(program):
    limpar estado temporário de concorrência, fork/join e canais
    PASSO 1 — coletar declarações:
        registrar canais, funções e classes no escopo global
        registrar atributos e métodos em escopos de classe
        verificar ciclos de herança e compatibilidade de sobrescritas
    PASSO 2 — validar cada declaração/instrução em ordem:
        ao entrar em bloco, função, classe, laço ou ramo concorrente:
            entrar no escopo e salvar contexto
        inferir tipos e validar declarações/atribuições
        validar chamadas, argumentos, condições e retornos
        validar membros herdados e acessos a índices
        em PAR/fork: marcar escopos concorrentes; verificar escritas externas
        registrar forks e chamadas de canal para validar joins e assinaturas
        ao sair, restaurar escopo e estado anterior
    validar compatibilidade de aridade e tipos entre envios/recebimentos
    devolver sucesso se o coletor não contém erros
```

**Inferência de tipos (2026.2, mais precisa que 2026.1):** literais inteiros → `int`, decimais → `float`; aritmética promove para `float` se algum operando for `float`; `/` sempre `float`; `int` → `float` implícito, `float` → `int` só explícito; `'a'`/`texto[i]` → `char`; `char`/`string` compatíveis (no C++ ambos `std::string`); operadores lógicos/relacionais → `bool`; `new Classe` → `Classe`; `self` → classe atual. Tipos desconhecidos (`unknown`) são tratados como compatíveis quando não se pode concluir estaticamente; erros conhecidos são **acumulados**, não exceções.

**Diagnósticos:** `Diagnostic(phase, message, line, column, length, severity)` imutável; `DiagnosticBag` acumula e formata com contexto:

```
erro semântico [12:5]: Variável 'x' não declarada.
  12 | x = 42;
     | ~~~~~
1 erro(s), 0 aviso(s)
```

Fases: `lexical`/`syntax`/`semantic`/`generation`/`execution` (+ aliases PT `léxico`/`sintático`/etc.). Exportação `to_dict()`/`to_json()` alimenta a IDE e a API. É o mesmo `DiagnosticBag` compartilhado entre léxico/parser/semântico via `set_diagnostic_collector` no `CControle`; no fluxo concreto `compile_file(...)` um coletor único é passado a `Lexer`, `Parser` e `SemanticAnalyzer`.

Regras verificadas (exemplos de mensagem): variável não declarada / já declarada no escopo; tipo incompatível em atribuição; função/classe/método/atributo/superclasse inexistente; aridade errada; `break`/`continue` fora de laço; escritura externa em ramo `PAR`/`fork`; `join` sem `fork` (ou `fork` sem `join`) no mesmo bloco; `send`/`receive` sem `c_channel` declarado; dois computadores iguais em `c_channel`.

Recuperação sintática: ao erro, avança até `NEWLINE`/`DEDENT`/`EOF` sem consumir o resto do bloco válido (ver §4).

---

## 6. Tabela de símbolos

**Implementação:** `components/tabela_simbolos/c_tabela_simbolos.py` (`TabelaSimbolos.enter_scope/exit_scope/define/lookup/lookup_local/lookup_member/export`), contrato `ITabelaSimbolos` em `components/interfaces.py`. Documento complementar: `docs/pseudocodigo_tabela_simbolos.md`.

A tabela é uma **árvore de escopos** (`_Scope{name, path, parent, symbols, children}`) com **pilha de escopos ativos** (`_scopes`) e índice de **todos os escopos criados** (`_all_scopes`). Escopos encerrados permanecem registrados para exportação e consulta de membros de classe, mas só escopos ativos participam da busca lexical.

```
PSEUDOCÓDIGO DA TABELA DE SÍMBOLOS (resumo fiel a c_tabela_simbolos.py)

inicializar:
    criar escopo global; pilha_atual = [global]; registrar árvore

entrar_escopo(nome):
    pai = topo(pilha_atual); criar filho; caminho único sob pai
    registrar na árvore e no índice global; empilhar escopo

sair_escopo:
    se mais de um escopo ativo: desempilhar (mantém registrado)

declarar(simbolo):
    escopo = topo(pilha_atual)
    se nome já existe em escopo: retornar falso
    registrar símbolo com caminho de escopo atual; retornar verdadeiro

buscar(nome):
    para escopo em pilha_atual, do mais interno ao global:
        se nome existe: retornar símbolo
    retornar nenhum

buscar_local(nome): retornar símbolo no topo ou nenhum

buscar_membro(classe, nome):
    visitadas = {}
    enquanto classe existe e não visitada:
        marcar visitada; procurar nome no escopo de membros
        se encontrado: retornar símbolo
        classe = superclasse registrada
    retornar nenhum

exportar: serializar cada escopo registrado, seu pai e seus símbolos
```

**Detalhes atuais:** `enter_scope(nome, reuse=False)` gera caminho único (`global/bloco#2`, etc.) e permite reuso para blocos de classe; `register_class_scope`/`lookup_method`/`lookup_attribute` apoiam herança; `Symbol` carrega `name`, `type`, `params`, `parameter_types`, `signature`, `required_params`, `attributes`, `methods`, `category` (`variable`/`function`/`method`/`class`/`channel`...), `scope`, `line`, `parent_class` (com aliases PT `nome`, `tipo`, `escopo`, `linha`, `assinatura`, `classe_pai`). A API web expõe em `POST /api/symbols` (`{code}` → `symbols` + diagnósticos).

---

## 7. Geração de código intermediário TAC

**Implementação:** `components/tac/c_gerador_tac.py` (`TACGenerator.generate/generate_text`), execução em `components/tac/executor_tac.py`, otimizações em `components/tac/otimizador_tac.py`, utilidades em `components/tac/instrucoes.py`. O TAC é **texto** de três endereços escrito em `output/*.tac` e também usado para o lowering C++; o executor interpreta o texto (filas FIFO em memória, ramos intercalados sequencialmente — simulação, não paralelismo real).

Expressões binárias/unárias são quebradas em temporários `t0, t1, …`; fluxo usa rótulos `WHILE_n`, `ENDWHILE_n`, `ENDIF_n`, `LOOP/PARTITION_LOOP/PARTITION_END/END` (no caso T7) e pilhas de `break`/`continue`. Objetos usam `alloc`, `getfield`, `setfield` (chamada de método = função com objeto como 1º argumento). Blocos paralelos delimitam ramos com `par_begin`/`par_end`; `send`/`recv` representam canais no TAC.

Exemplo (semântica de `var soma = a + b * c;`):

```
t0 = b * c
t1 = a + t0
soma = t1
```

Enquanto (`while`):

```
WHILE_0:
ifFalse cond goto ENDWHILE_1
...
goto WHILE_0
ENDWHILE_1:
```

**Equivalência com o PDF (T7):** a comparação é por **equivalência de execução**, não texto idêntico — gerador usa rótulos genéricos/temporários tipados, o PDF usa nomes semânticos e `if ... goto`. Ambos produzem `Array ordenado: [5, 12, 33, 61, 98]`. Mapa completo em `docs/relatorio/RELATORIO-TESTES-2026.2.md` §5.3; referência textual conferida em `tests/integracao/expected_results/t07_quicksort_iterativo_tac.expected.tac` (só rótulos estruturais por `tests/programas/test_t07.py`).

**Otimizador:** `TACOptimizer` aplica otimizações simples (níveis `O0`–`O3`). Na IDE, **Otimizar TAC** compara original × otimizado lado a lado (`POST /api/otimizar`). O nível escolhido no seletor "TAC" do editor também é enviado em `POST /api/executar` e `POST /api/baixar-exe`: com nível diferente de `O0`, o C++ é regenerado a partir do TAC otimizado antes de compilar.

---

## 8. Geração de código C++ e compilação com GCC

**Implementação:** `components/codegen/c_gerador_cpp.py` (transpila TAC → C++17) + `components/codegen/driver_gcc.py` (`compilar`/`executar`). O driver grava `.cpp` e executável em `output/` (ou `-o`), com `-std=c++17` e `-O2` por padrão (`-O0`–`-O3` no CLI/IDE).

```
CppCodeGenerator.generate(tac, output_name) → .cpp
DriverGCC.compile(cpp_path, saida, nivel_otimizacao="O2") → bool
DriverGCC.execute(saida) → (stdout, exit_code)
```

Comando efetivo (Linux): `g++ -std=c++17 -Wall -O2 saida.cpp runtime/canal.cpp runtime/paralelo.cpp runtime/worker.cpp -I runtime -o saida -pthread`. No Windows: `-lws2_32` e `.exe`. `CXX` configura o compilador se não for `g++`. Diagnósticos do `g++` são remapeados para linhas MiniPar via marcadores `// MiniPar line N` emitidos no C++.

Mapeamento MiniPar → C++ (atualizado para 2026.2):

| MiniPar | C++ gerado |
|---|---|
| `class C { ... }` | `struct/class C` com campos `field_0...` e métodos |

| MiniPar | C++ gerado |
|---|---|
| `new C(args)` | construtor/`alloc` + `getfield`/`setfield` no TAC |
| `obj.metodo(args)` | chamada com objeto como 1º argumento |
| `obj.campo`, `this.array[i] = v` | `__minipar_array_set(self->field_0, i, v)` (correção de cópia — T6) |
| `par { ... }` | `minipar_parallel(tasks, core_indices, task_names)` com threads |
| `fork t em core(N):` | thread com afinidade à N-ésima CPU permitida (`taskset`-aware); sem `core()` o SO escolhe; falha de afinidade avisa e executa sem |
| `fork t em host("pc2"):` | chamada remota ao `minipar_worker` via canal tipado |
| `c_channel` + `send`/`receive` | `MiniParChannel` (`ouvir`/`conectar`/`enviar`/`receber` + adaptadores `sendData`/`receiveData`) |
| `list`/`matrix` | `std::vector<...>` (ex: `__make_matrix`) |
| `int`/`float`/`bool`/`string`/`char` | `int64_t` / `double` / `bool` / `std::string` (`char` = string de 1) |

**Runtime (`runtime/minipar_rt.hpp`, 636 linhas + `canal.cpp`/`paralelo.cpp`/`worker.cpp`):**

- **Canais TCP tipados:** campo = `(quantidade, tipo, tamanho, bytes)`, big-endian, inteiros 64 bits, `float`/`double` IEEE-754 64 bits, `bool` 1 byte, strings UTF-8. `enviar<T...>()`/`receber<T...>()` validam quantidade/tipos; `setTimeout` (padrão 5 s; `MINIPAR_TIMEOUT_MS` no worker; `MINIPAR_CHANNEL_TIMEOUT_MS` altera o padrão dos canais — o executor da Conexão usa 10 min porque o outro lado espera o usuário digitar; a espera pela conexão do par (`ouvir`/`conectar`) nunca passa de 60 s, para que falhas de rede apareçam logo); `static_assert` por nº de campos; falhas → `std::runtime_error`. Limites `max_field_size=16 MiB`, `max_frame_size=64 MiB`, `max_field_count=1024`, mutexes de envio/recepção. `sendData`/`receiveData` mantidos como adaptadores de mensagem string.
- **`c_channel` lógico + `minipar.hosts`:** `computador_1 = 127.0.0.1:5001`. Linhas vazias/`#` ignoradas, porta 1–65535. Processo `--como computador_1` conecta a `computador_2`; `--como computador_2` faz `ouvir`. Papel inexistente → erro com nomes válidos. `--hosts caminho` troca o arquivo. Trocar IP:porta não exige recompilar.
- **Paralelismo:** `minipar_parallel` com `minipar_output_mutex` para `print` (saída intercalada sem corrupção), `random` LCG thread-local (`state*1103515245+12345 & 0x7fffffff`), `agora()`/`clock()` monotônico, afinidade `core(N)` e `MINIPAR_TRACE_CORES=1` + `benchmarks/benchmark_fork_core.py` (4 tarefas CPU-bound, speedup >2,5× em ≥4 CPUs).
- **`range`:** `__minipar_range` devolve `std::vector<double>` (mesmo tipo de `list` no código gerado); antes devolvia `std::vector<int>` e `for x in range(n)` não compilava no C++.
- **Helpers:** `__parse_ints` retorna `std::vector<double>` (o tipo `list` mapeia para `std::vector<double>` em `c_gerador_cpp.py:108`; não há correção para `int64_t`), `input`, `__minipar_format_float`, `__to_string`/`__minipar_concat`, `__minipar_to_int/float`, `__minipar_len/sum/max`, `__minipar_divide` (zero → `runtime_error`), string/arquivo (`__minipar_char_at/substring/is_digit/is_alpha/is_space/is_number`, `__minipar_write_file`), `__minipar_array_at/set` com checagem de limites e `__array_pop`, `__make_matrix`, `minipar_init`/argumentos `--hosts/--como/--porta`, registro de tasks `minipar_register_task`/`minipar_run_worker_if_requested`/`minipar_remote_call`.

**Fork remoto:** o codegen gera junto ao executável `minipar_worker` (`.exe` no Windows) com os mesmos handlers. Args/resultados `int/float/bool/string` por valor pelo protocolo do canal; cada subtarefa tem nome estável. Iniciar `minipar_worker --porta 5001` por máquina, cliente com `fork t em host("computador_2")`, `MINIPAR_TIMEOUT_MS=5000` por padrão. Teste automatizado valida 2 subtarefas remotas em localhost; validação em 2 máquinas reais é pendência de T41 (ver `docs/rede.md`).

**Segurança de memória (issue #45):** pilha do T7 dimensionada; leitura/escrita fora dos limites falham explicitamente com mensagem e código não-zero (via `__minipar_array_at/set`); CI exige ASan/UBSan no Linux.

---

## 9. Interface web (IDE didática)

**Implementação:** Flask em `interfaces/web/api.py`, páginas em `interfaces/web/frontend/templates/` (`base.html` + `painel/benchmark/resultados/configuracoes/editor`), assets em `static/` (CSS próprio, sem build), CodeMirror 5.65.13 local em `static/vendor/codemirror/5.65.13/` (`codemirror.min.css/js`, `mode/clike`, `theme/dracula` legado, `LICENSE` MIT), fontes de pilha do sistema, ícones em sprite SVG embutido. **Nenhuma requisição externa** (logo textual `MP`/`IC`). Para atualizar o CodeMirror, substituir os arquivos no mesmo caminho e manter `LICENSE`.

| Rota | Função |
|---|---|
| `GET /` | Painel — configura benchmark e navega para `/benchmark?autorun=1` (não executa direto) |
| `GET /benchmark` | Execução, progresso (polling), métricas, export CSV, histórico de até 10 medições, tabela paginada (janela de 5 páginas / 15 linhas) |
| `GET /resultados`, `/resultados/<run_id>` | Histórico do compilador com visão 7 fases (Código, Tokens, AST, Símbolos, TAC, C++, Execução) readonly; `compileResult` restaurável; fallback C++ salvo; `/#result-detail` redireciona para mais recente |
| `GET /configuracoes`, `/editor`, `/pipeline` | Configurações (persistidas em `localStorage`), IDE e guia didático |
| `GET /ajuda` | Página de Ajuda (item "Ajuda" no menu lateral): uso do editor, linguagem (inclui `do … while`), paralelismo, canais/dois computadores e leitura de erros |
| `GET /conexao` e `/api/conexao/*` | Hub de Conexão (ver §9.1): hubs, salas, chat, convites e execução em par |
| `POST /api/baixar-exe` | `{code, filename?, optimization_level?}` → compila (mesmo caminho do executar, com TAC otimizado se pedido) e devolve o executável como anexo (`.exe` no Windows). Erros de compilação → 422 com `diagnosticos`; nível inválido/entrada inválida → 400 |
| `POST /api/compilar` | Recebe `{code}` → `tokens`, `ast`, `tabela_simbolos`, `tac`, `cpp`, `tac_origin`, `cpp_origin`, `diagnosticos`; mapas relacionam `generated_line`→`source_line` (highlight ao hover/teclado) |
| `POST /api/executar` | Mesmo código + `inputs` opcional → compila e executa em subprocesso → `stdout`, `stderr`, `exit_code`, `tempo` (corte 30 s ou 1 MiB combinados) |
| `POST /api/symbols` | `{code}` → `symbols` + diagnósticos |
| `POST /api/benchmark`, `GET /api/hardware` | Job assíncrono (hardwares, params limitados, polling, mediana/speedup/eficiência, CSV); workload gerado via pipeline TAC/C++ MiniPar |
| `POST /api/compile` | Rota legada (IDE atual): compila e executa em uma chamada |
| `POST /api/get_test`, `/api/test_spec`, `/api/session/create` | Carregam de `programas_teste/` só por **caminho relativo dentro do dir**; absolutos/`..`/vazio/não-texto/inexistente → 400/404 |

Backend de benchmark (`interfaces/web/benchmark.py`): hardware, params limitados, jobs assíncronos, polling, medição monotônica dentro das tarefas, mediana/speedup/eficiência e CSV. A carga ajusta atividades quando total >2.000;

partição de núcleos descoberta balanceada. A comparação `original vs otimizado` (`POST /api/otimizar`) é didática; o nível de TAC do editor, porém, vale para a execução e para o executável baixado (ver §7).

**Setup:** `uv sync --extra dev`, `python main.py` (menu: `[1]` terminal, `[2]` web em `http://127.0.0.1:5000`), ou `minipar compilar programa.mp -O2 -o output/prog --executar`. Validação offline: desativar rede no navegador e rodar testes de integração.

### 9.1 Hub de Conexão e execução em dois computadores

**Implementação:** `interfaces/conexao/` (servidor `ServidorHub`, cliente `ClienteHub`, protocolo JSON por linha sobre TCP), `interfaces/web/conexao_api.py` (blueprint Flask) e `interfaces/web/conexao_execucao.py` (`ExecutorConexao`: compila o programa, prepara e inicia o processo local). No front-end: `conexao.js` (página `/conexao`), `conexao_status.js` (estado compartilhado), `conexao_global.js` (chip de status, convites e chat em todas as páginas) e `multi_exec.js` (terminal de execução em par no editor).

- **Hubs e salas:** a página `/conexao` lista os hubs encontrados na rede (descoberta por broadcast UDP na porta 5011 e lista manual de servidores). Ao entrar em um hub, o usuário cai no **lobby (sala pública)**, com chat livre, e vê a lista de salas privadas de **até 2 pessoas**, com cadeado nas protegidas por senha. Ao fechar uma conexão a partir do lobby, o par entra automaticamente em uma sala privada.
- **Ciclo de vida da sala:** só o dono (quem criou a sala ou enviou o pedido de conexão) dissolve a sala ao sair; se o outro sair, a sala continua aberta aguardando alguém. Salas ociosas fecham sozinhas após 2 min de inatividade, com aviso antes.
- **Status em todas as páginas:** um chip na barra superior informa se há conexão com um hub, qual sala e qual parceiro (ou "Conexão perdida"), em todas as telas (painel, editor, conexão, resultados, benchmark, configurações).
- **Execução em par pelo editor:** ao executar um programa que tem par cliente/servidor (por exemplo `t01_calculadora_cliente.mp`), o editor envia um **convite** ao parceiro da sala privada ou, se o usuário estiver só no lobby, a todos no lobby (o primeiro a aceitar fecha o par). Quem aceita é levado ao editor com o programa do outro lado carregado. Nos dois lados aparece um **terminal de altura fixa**, no lugar das "Entradas do programa", com entrada interativa linha a linha; as fases (Código, Tokens, AST, Símbolos, TAC, C++, Execução) continuam abaixo. O hub coordena preparo e início (o servidor escuta primeiro) e encerra os dois lados se um deles parar.
- **Rede virtual (Radmin, Hamachi):** a descoberta envia o pedido por cada interface local, com broadcast para as redes /24, /16 e /8 (o Radmin usa `26.0.0.0/8`), e o IP anunciado ao parceiro é o da rede pela qual o hub é alcançado, não o da rota padrão.
- **Exemplos de pares:** `programas_teste/t01_multiterminal/` (calculadora) e `programas_teste/dois_computadores/` (adivinhação, estoque, conversor de unidades, cofre com senha, pedra/papel/tesoura e estatísticas), todos com `_cliente` e `_servidor`. No jogo de adivinhação e no cofre, quem está no servidor escolhe o número/senha, de modo que o servidor também participa do programa.

---

## 10. Product Backlog e Sprint Backlogs

Snapshot **08/11/2026** do GitHub de `johnUFAL/minipar-improvement-2026.2` — fonte completa: `docs/backlog.md`. Issues abertas = **Em andamento** (estado de execução não inferido); fechadas = **Feito**. Prioridades: labels `P0/P1/P2`. Todas as 50 issues associadas a exatamente um dos 9 milestones no snapshot.

### Product Backlog

| Issue | P | Sprint | Status | Item |
|---|---|---|---|---|
| #1 | P0 | 0 | Feito | [T01] Reestruturar fork, ambiente e CI |
| #2 | P0 | 0 | Feito | [T02] Gramática EBNF 2026.2 |
| #3 | P0 | 0 | Feito | [T03] Contratos e modelos compartilhados |
| #4–#6 | P1 | 0 | Feito | [T04–T06] UML casos de uso, componentes, classes |

| Issue | P | Sprint | Status | Item |
|---|---|---|---|---|
| #7 | P0 | 1 | Feito | [T07] Erros e reporter |
| #8 | P0 | 1 | Feito | [T08] Léxico com comentários e INDENT/DEDENT |
| #9 | P0 | 1 | Feito | [T09] Tabela de símbolos com escopos |
| #10 | P0 | 2 | Feito | [T10] AST e Visitor |
| #11–#13 | P0 | 2 | Feito | [T11–T13] Sintático I, II (OO), III (paralelismo/canais) |
| #14 | P0 | 3 | Feito | [T14] Semântico I: tipos/escopo/funções |
| #15–#16 | P1 | 3 | Feito | [T15–T16] Semântico II (classes/herança), III (concorrência/canais) |
| #17 | P0 | 4 | Feito | [T17] TAC I e executor |
| #18 | P0 | 4 | Feito | [T18] TAC II: objetos e paralelas |
| #19 | P2 | 4 | Feito | [T19] Otimizações TAC |
| #20 | P0 | 5 | Feito | [T22] Runtime C++ base |
| #21 | P0 | 5 | Feito | [T20] Gerador C++ |
| #22 | P0 | 5 | Feito | [T21] Driver GCC |
| #23 | P0 | 6 | Feito | [T23] Canais TCP tipados |
| #24 | P0 | 6 | Feito | [T24] Topologia e cliente/servidor |
| #25 | P0 | 6 | Feito | [T25] PAR com threads |
| #26 | P1 | 6 | Feito | [T26] Fork/Join local em núcleo |
| #27 | P1 | 6 | Feito | [T27] Fork/Join remoto |
| #28 | P0 | 7 | Feito | [T28] Controle e CLI |
| #29 | P0 | 7 | Feito | [T29] API do pipeline |
| #30 | P1 | 7 | Feito | [T30] Interface web didática |
| #31 | P1 | 7 | Feito | [T31] Visualização AST |
| #32 | P0 | 8 | Em andamento | [T32] Teste 1: calculadora cliente/servidor |
| #33 | P0 | 8 | Em andamento | [T33] Teste 2: fatorial/Fibonacci em PAR |
| #34 | P1 | 8 | Em andamento | [T34] Teste 3: neurônio com classes |
| #35 | P1 | 8 | Em andamento | [T35] Teste 4: rede XOR |
| #36 | P1 | 8 | Em andamento | [T36] Teste 5: recomendação |
| #37 | P1 | 8 | Em andamento | [T37] Teste 6: quicksort com classes |
| #38 | P0 | 8 | Em andamento | [T38] Teste 7: quicksort iterativo |
| #39 | P1 | 8 | Em andamento | [T39] Benchmarks 8–10 |
| #40 | P0 | 8 | Em andamento | [T40] Testes de integração |
| #41 | P1 | 8 | Em andamento | [T41] Execução, telas, validação em 2 máquinas |
| #42 | P1 | 8 | Em andamento | [T42] Pseudocódigos léxico/sintático/semântico/símbolos |
| #43 | P1 | 8 | Em andamento | [T43] Backlogs |

| Issue | P | Sprint | Status | Item |
|---|---|---|---|---|
| #44 | P0 | 8 | Em andamento | [T44] Relatório final e release |
| #45 | P0 | 5 | Em andamento | [T45] Corrigir corrupção de memória do T7 e limites de array |
| #46 | P1 | 8 | Em andamento | [T46] Regressão T1–T10 |
| #47 | P1 | 7 | Em andamento | [T47] Backend de benchmark com métricas reais |
| #48 | P1 | 7 | Em andamento | [T50] Empacotar assets locais |
| #49 | P1 | 7 | Em andamento | [T48] Painel/Resultados/Configurações em templates Flask |
| #50 | P0 | 7 | Em andamento | [T49] Migrar IDE/terminal/sessões para o novo layout |

> **Soma:** 31 Feito (Sprints 0–6 + metade do 7) e 19 Em andamento (Sprint 8 inteiro + caudas do 5 e 7). Não inferir não-início só por issue aberta.

### Sprint Backlogs (incrementos registrados)

- **Sprint 0 — Fundação e arquitetura (#1–#6, Feito):** estrutura, gramática, contratos/modelos, UML.
- **Sprint 1 — Léxico e símbolos (#7–#9, Feito):** diagnósticos, lexer com indentação, tabela com escopos.
- **Sprint 2 — Sintático e AST (#10–#13, Feito):** AST/Visitor, parser de comandos/OO/paralelismo/canais.
- **Sprint 3 — Semântico (#14–#16, Feito):** tipos/funções, classes/herança, concorrência/canais.
- **Sprint 4 — Intermediário (#17–#19, Feito):** geração/execução TAC (expressões, fluxo, objetos, paralelo) + otimizações.
- **Sprint 5 — C++/GCC (#20–#22 Feito; #45 Em andamento):** runtime, gerador C++, driver GCC; #45 localmente implementada (pilha dimensionada, limites, teste sanitizado, CI ASan/UBSan) — falta anexar/confirma job Linux.
- **Sprint 6 — Paralelismo e canais (#23–#27, Feito):** TCP tipado, topologia, PAR, fork/join local/remoto.
- **Sprint 7 — Interfaces (#28–#31 Feito; #47–#50 Em andamento):** CLI, API, IDE didática, AST; #47 backend e #49 (T48) telas Flask localmente implementados (`base.html`, rotas próprias, histórico 10 medições, janela 5 páginas, persistência `localStorage`); migração IDE/sessões (#50/T49) e assets 100% offline (#48/T50) pendentes.
- **Sprint 8 — Testes, docs e entrega (#32–#44 + #46, Em andamento):** sem issues fechadas no snapshot; pseudocódigos, backlog e relatório atualizados, mas revisão humana, Project (`read:project` ausente), matriz dos 17 itens, checklist oficial, clone limpo e release pendentes.

**Plano priorizado** (detalhes e plano passo a passo por issue em `ANALISE-ISSUES-E-PLANO.md`):

1. Segurança de memória T7 (#45, P0) — implementada localmente, falta evidência do job Linux no CI.
2. Consolidar integração (#40, P0) e programas P0 (#32, #33, #38) — build/link do runtime e casos reproduzíveis.
3. Artefatos P1 do Sprint 8: regressão T1–T10 (#46), programas T3–T6 e benchmarks T8–T10 (#34–#37, #39), pseudocódigos (#42), evidências (#41).
4. Entrega e relatório (#43, #44) — backlog com 9 sprints feito; falta board, matriz 17 itens, clone limpo, PDF/tag/release.
5. Benchmark (#47) — API/painel prontos; falta confirmar job completo e equivalência do workload com T8–T10.

### 10.1 Modelagem UML

Fonte: `docs/uml/*.puml` (+ `.png`). Diagramas atualizados para 2026.2:

- **`casos_de_uso.puml` (T04):** atores `Programador`, `Aluno`, `Computador remoto`; casos `Compilar programa` (base), `Ver erros`, `Escolher nível de otimização`, `Ver tokens/AST/símbolos/TAC/C++`, `Executar localmente` e `Executar em dois computadores` (com nota: hoje sessão multi-terminal; alvo `fork em host` + `c_channel`). Relações `<<include>>` (executar exige compilar) e `<<extend>>` (erros/opt e inspeções condicionais).
- **`componentes.puml` (T05) e `classes.puml` (T06):** pacotes `Contratos ABC`, `Pipeline injetável` e `Implementações concretas`/`Componentes concretos`; artefatos `Tokens`, `AST (Program)`, fonte `.mp`, `.tac`, `.cpp`, executável; pacote `Runtime C++`. **Nota:** `classes.puml` está defasado em relação ao código — lista

`TokenType` antigos (`NUMBER`, `FUNC`, `THIS`, `SEND`, `RECEIVE`, `MATRIX`, `SEMICOLON`), afirma que `TabelaSimbolos` não implementa os contratos (ela implementa `ITabelaSimbolos` em `c_tabela_simbolos.py:88`) e que `DiagnosticBag` não está ligado ao `SemanticAnalyzer` (está em `c_semantico.py:342-343`). As tabelas `TokenType`/`Token`/`Diagnostic`/`Symbol` e os fluxos `Lexer→Parser→Semantic→TAC/Cpp→DriverGCC→Runtime` precisam de atualização.

- **`ast_expressoes/acessos/dados_io/comandos/fluxo/declaracoes.puml` (T10):** nós por grupo (§4), base `ASTNode(line,column,tipo_inferido)` sem classes intermediárias `Expr`/`Stmt`; `AttributeDecl` especializa `VarDecl`, `MethodDecl` especializa `FuncDecl`; `ASTVisitor` visita filhos em tuplas/listas, `ASTPrinter` a estende.
- **`tac_runtime.puml` (T18) e `tac_planejado.puml` (futuro):** `TACGenerator`/`TACExecutor`/`CppCodeGenerator`/`DriverGCC` + runtime; instrução TAC tipada ainda planejada, hoje TAC é texto. **Atenção:** `tac_runtime.puml` ainda descreve `CppCodeGenerator.generate(program, output_name)` traduzindo AST; o código real é `generate(tac, output_name, nivel_otimizacao="O2")`, que traduz TAC e compila via `DriverGCC` (`c_gerador_cpp.py:1036-1051`).

---

## 11. Testes de integração e programas de exemplo (T1–T11)

A validação passa por **lexer → parser → semântico → TAC → C++ → `g++` → execução**, via runner `scripts/run_testes_2026_2.py` (CLI) e suíte `pytest` (`tests/programas/` com comparadores por programa + `tests/integracao/test_pipeline_boundaries.py` com positivo/negativo por fronteira). O menu interativo só **monitora** sessões; testes multi-terminal usam o runner scriptado.

**Resultado geral (execução original 06/11/2026, Windows 10, Python 3.11.15, GCC 16.2.0 MSYS2 UCRT64): 11/11 passaram** (~1 min 34 s de soma dos tempos versionados no JSON, com compilação). A execução original relatada como **216 passed, 1 skipped (taskset Linux), 40 subtests em 245,57 s** não possui artefato versionado no repositório que a comprove. A revalidação de 08/11/2026 foi feita na branch **`fix/quest_docs_minipar_2026`** (Python 3.12.10, GCC 16.2.0), não na `main`: **247 passed, 8 failed, 8 skipped, 89 subtests, cobertura 81,03% (mínimo 80%)** — falhas de T1 (#32) e dos assets offline (#48–#50) naquela branch, mais `ruff` em `scripts/run_multiterminal.py` (`expected_text` indefinido). `mypy` passou. Na `main` atual os assets do CodeMirror estão versionados localmente (`interfaces/web/frontend/static/vendor/codemirror/`), portanto a falha de CDN não se aplica a este checkout.

**Revalidação de 10/11/2026 (checkout atual, Windows, Python 3.15.1, GCC 15.2.0 MinGW-w64):** `ruff check .` e `mypy` passaram. A suíte `pytest` completa, antes das últimas alterações (`do … while`, download do executável, página de Ajuda e correção de `range`), terminou com **339 passed, 3 skipped, 1 deselected, 141 subtests em 18 min 52 s**; o teste desselecionado é `test_session_endpoints_manage_multi_terminal_lifecycle`, que já falhava antes das mudanças do hub e não tem relação com elas. Depois das últimas alterações, passaram os 200 testes unitários e os testes novos (`test_do_while.py`, `test_lacos_execucao.py`, `test_download_exe.py`); a suíte completa **não foi reexecutada até o fim** após elas, e a cobertura (mínimo 80% no CI) não foi medida nesta execução (`pytest-cov` ausente no ambiente local).

| Teste | Descrição | Modo | Tempo (JSON versionado) |
|---|---|---|---|
| T1 | Calculadora cliente/servidor | 2 processos, canal lógico | 15,30 s |
| T2 | Fatorial e Fibonacci em `par` | local | 7,61 s |
| T3 | Neurônio com classes | local | 7,30 s |
| T4 | Rede neural XOR | local | 7,31 s |
| T5 | Sistema de recomendação | local, 16 entradas | 7,48 s |
| T6 | Quicksort com classes | local, 1 entrada | 7,35 s |
| T7 | Quicksort iterativo | local | 7,49 s |
| T8 | Avaliação em 4 núcleos | `fork/core` | 8,18 s |

| Teste | Descrição | Modo | Tempo (JSON versionado) |
|---|---|---|---|
| T9 | 5 núcleos + Mandelbrot | `fork/core` | 8,57 s |
| T10 | 6 núcleos + 2 fractais | `fork/core` | 8,77 s |
| T11 | Minicompilador ARMv7 | local | 9,12 s |

Entradas fixas: T1 `+ 7 5`; T5 16 linhas `1 0 0 0 0 1 0 0 0 1 0 0 1 0 0 0`; T6 `10 -3 -40 80 70 -100`; T2–T4, T7–T11 sem entrada. Saídas T2–T7 congeladas em `tests/integracao/expected_results/` com normalização CRLF→LF. JSON com checks por teste: `docs/relatorio/resultados/testes_2026_2/testes_2026_2.json`; logs por teste em `testes_2026_2/t*.log`.

### 11.1 Testes obrigatórios

- **T1 — calculadora multi-terminal (protocolo):** par `programas_teste/t01_multiterminal/` (`t01_calculadora_servidor.mp`/`_cliente.mp`) com `c_channel dados computador_1 computador_2`, payload fixo `(string,float,float,float)`: requisição `(op,v1,v2,0.0)`, resposta `(status,resultado,0.0,0.0)`, `status="OK"` / `"Erro: divisao por zero."` / `"Erro: operacao invalida."`. Servidor em loop até `"sair"`; erro não derruba o serviço. Runner compila os dois, aloca porta livre, escreve `minipar.hosts`, inicia o servidor, aguarda `LISTENING`, executa o cliente e valida saídas. Exemplos:

  ```
  python scripts/run_multiterminal.py --scenario all
  python scripts/run_multiterminal.py --requests "/,5,0;+,12,30;sair"
  ```

  O `t01_calculadora.mp` original (com `c_channel("127.0.0.1",5101)`) fica como referência histórica, não executável sozinho. Evidência:

  ```
  Servidor recebeu: + 7.0 5.0 / Servidor respondeu: OK 12.0
  Cliente enviou: + 7.0 5.0 / Resultado: 12.0
  ```

- **T2 — fatorial/Fibonacci em PAR:** dois `SEQ` filhos; fatorial iterativo, Fibonacci com sequência validada por ramo (ordem global não determinística); sobreposição medida em 20 execuções.
- **T3 — neurônio com classes:** `class Neuronio` + `init` com padrão, `new`, `self.*`; linhas-chave das iterações 1, 50, 51 e conclusão (não byte a byte).
- **T4 — rede XOR:** 3 neurônios ocultos, 20.000 épocas, pesos pseudossorteados; comparador exige exatamente 4 previsões finitas em `[0,1]` (não compara valores). Benchmark §5.2: C++ `-O2` mediana **0,0147 s** vs Python **0,2292 s** = **15,62×** (`t04_tempo.json`, 5 amostras).
- **T5 — recomendação:** rede determinística; golden confere as **12 recomendações** na ordem e acentos UTF-8 (`Máquina de lavar`, `Não-ficção`, `Ficção científica`).
- **T6 — quicksort com classes:** método recursivo `_quicksort`; entrada do PDF + stdin cobrindo `1 a 3` (reentrada `dividir`/`eh_inteiro`), entrada vazia e vetor unitário.
- **T7 — quicksort iterativo/TAC:** programa + saída `Array ordenado: [5, 12, 33, 61, 98]`; TAC por equivalência (§7).
- **T8–T10 — fork/join em núcleos:** T8 160 atividades em 4 núcleos; T9 4 avaliadores + Mandelbrot (núcleo 5); T10 6 núcleos + Sierpinski ASCII (núcleo 6). Runner valida marcos (total, cabeçalho, nº de núcleos, fractais), não tempos exatos. `random_seed(t)` por atividade garante mesmo trabalho seq/par e checksums iguais. Medição (Windows, 14 CPUs, `-O2`, mediana 5 amostras, base Python 1 amostra):

| Teste | Seq (s) | Par (s) | Speedup | Núcleos | Base Python speedup |
|---|---|---|---|---|---|
| T8 | 0,41 | 0,13 | **3,15×** | 4 | 1,12× |
| T9 | 0,40 | 0,13 | **3,08×** | 5 | 1,19× |
| T10 | 0,40 | 0,13 | **3,08×** | 6 | 1,29× |

T8 cumpre **>2,5×**; Python fica 1,1–1,3× por *spawn* `multiprocessing` no Windows e trabalho curto. No baseline Python do T10 o Sierpinski usa Turtle/Tk e não roda headless — etapa desativada e registrada.

- **T11 — minicompilador ARMv7:** gera tokens/AST/TAC/Assembly/tabela/resultados. Esperados: `((2+4)*7)-(5*4)=22.0`, `((10-8*7)+4)*2=-84.0`, `22.0`. Arquivos `programas_teste/teste11/expr1-3.arm` montam com `clang --target=armv7-none-eabi -x assembler -c …`; execução no CPUlator (`r8`) permanece manual.

### 11.2 Testes adicionais e inválidos

- `teste_adicional1_paralelismo_3_computadores.mp` — `PAR` com `send`/`receive` em 3 canais; `fork ... em host("computador_2")`.
- `teste_adicional2_tapete_sierpinski.mp` — regra dos dígitos na base 3 sem `turtle`.
- `teste_ascii_art_divertidapkrl.mp`, `teste_bnf_controles_input.mp`, `teste_builtins_string.mp` — matriz ASCII, controles+Bnf com `input`, builtins de string/arquivo.
- **Pares para dois computadores** (`programas_teste/dois_computadores/`, 6 pares): `tests/integracao/test_dois_computadores.py` compila e executa cada par de verdade (cliente e servidor em processos separados, com entradas de teste) e confere as saídas dos dois lados; `tests/integracao/test_conexao_hub.py` cobre salas, dono, inatividade, chat e convites.
- **10 programas inválidos** em `tests/programas/invalidos/` cobrindo léxico, sintático, semântico e concorrência, cada com mensagem esperada em `test_invalid_programs.py`. Fronteiras: léxico → parser, parser → semântico, semântico → TAC, TAC → C++ (`ValueError` em opcode desconhecido), C++ → GCC.

### 11.3 Tabela de estados dos testes

| Teste | Cenário coberto | Snapshot (JSON/relatório) |
|---|---|---|
| T1 | `c_channel`, `send`/`receive`, 2 processos | `Resultado: 12.0`, `Servidor recebeu: + 7.0 5.0` |
| T2 | `PAR`, racing de saída | `Fatorial de 10: 3628800`, `FIBONACCI 0/34` |
| T3 | Classe, `while`, aprendizado | iterações 1/51, `Valor desejado: 0`, convergência 51 iterações* |
| T4 | Funções, `exp`, backpropagation | 4 saídas em `[0,1]`; speedup 15,62× |
| T5 | Listas, indexação, recomendação | 12 recomendações, UTF-8 |
| T6 | OO, recursão, `parse_ints` | `[-100, -40, -3, 10, 70, 80]` |
| T7 | Funções, pilha manual, TAC | `[5, 12, 33, 61, 98]` |
| T8 | `fork/core`, benchmark | 160 atividades, 4 núcleos, speedup 3,15× |
| T9 | + Mandelbrot | 5 núcleos, `FRACTAL DE MANDELBROT` |
| T10 | + Sierpinski | 6 núcleos, `TAPETE DE SIERPINSKI` |
| T11 | MiniCompiladorARM, strings/arquivo | `22.0`, `-84.0`, `clang` monta 3 arquivos |
| Adicional 1 | 3 canais/host remoto | `fork em host` |

| Teste | Cenário coberto | Snapshot (JSON/relatório) |
|---|---|---|
| Adicional 2 | Recursão/matriz fractal | Tapete ASCII |

* O snapshot antigo cita "convergiu em 51 iterações"; a revalidação usa linhas-chave 1/50/51.

**Correções necessárias no backend (registradas no relatório de testes):** (1) `this.array[i]=v` copiava `self->field_0` em vez de `__minipar_array_set` no objeto — corrigido (T6); (2) **pendente:** `parse_ints` ainda retorna `std::vector<double>` (`runtime/minipar_rt.hpp:489`) e `list` mapeia para `std::vector<double>` (`c_gerador_cpp.py:108`); o teste `tests/unitarios/test_codegen.py:134` verifica apenas o nome do helper, não o tipo de retorno. O item 1 está coberto por `tests/unitarios/test_codegen.py`; o item 2 precisa de implementação e teste específicos.

**Como reproduzir:**

```
.\.venv\Scripts\python.exe scripts\run_testes_2026_2.py
.\.venv\Scripts\python.exe scripts\run_testes_2026_2.py --only t1,t11
.\.venv\Scripts\python.exe scripts\run_multiterminal.py --operacao + --v1 7 --v2 5
.\.venv\Scripts\python.exe -m pytest -q
```

---

# 12. Implementação

## 12.1 Tecnologias utilizadas

| Tecnologia | Uso |
|---|---|
| Python 3.11+ | Front/middle-end (léxico, sintático, semântico, TAC), CLI, menu, backend Flask |
| Flask 3.0 (`>=3.0`) | Servidor da IDE Web (API, templates, estáticos) |
| CodeMirror 5.65.13 (local) | Editor MiniPar/C++/TAC no navegador, sem CDN, com `LICENSE` MIT |
| C++17 (`-std=c++17`) | Linguagem alvo gerada (era C++20 em 2026.1) |
| `g++` (`CXX`) com `-O2` padrão (`-O0`–`-O3`) | Compilação do C++ gerado; `-pthread` (Linux), `-lws2_32` + `.exe` (Windows) |
| `std::thread`, `std::mutex`, `std::chrono` | `PAR`, `fork/core`, `minipar_parallel`, relógio monotônico, saída sincronizada |
| Sockets TCP | `MiniParChannel` em `canal.cpp` (`ouvir`/`conectar`/`enviar`/`receber`, frame tipado big-endian) |
| `pytest` + `pytest-cov`, `ruff`, `mypy` (extras `dev`) | Testes (cobertura mínima 80% no CI), lint (`E4/E7/E9/F`), tipos (`components,erros,interfaces`) |
| `playwright` (dev) | Testes de frontend/browser |
| `uv` (`uv sync/uv run`) | Gerenciamento de ambiente/dependências (`pyproject.toml`, `requirements.txt`) |
| Arquitetura armada em testes | Fakes no `test_pipeline.py`, `falsos.py`, ASan/UBSan no CI Linux |

Scripts de validação: `scripts/run_testes_2026_2.py`, `scripts/run_multiterminal.py`, `scripts/benchmark_t08_t10.py`, `scripts/benchmark_t04.py`, `benchmarks/benchmark_fork_core.py` e baselines Python `t04_rede_xor.py`, `t08_avaliacao_4_nucleos.py`, etc.

## 12.2 Fluxo de execução

**Via CLI `minipar` (instalado por `uv sync` ou `pip install -e .`):**

```
minipar compilar programas_teste/t07_quicksort_iterativo_tac.mp -O2 -o output/t07 --
executar

# Inspeção de fases, sem gerar executável:
minipar compilar programa.mp --tokens
minipar compilar programa.mp --ast
minipar compilar programa.mp --tac
minipar compilar programa.mp --emit-cpp

# Atalhos:
minipar tokens|ast|tac|cpp|build|run programa.mp
```

**Via `main.py`:**

```
python main.py                                # menu: [1] terminal, [2] web em
http://127.0.0.1:5000
python main.py programas_teste/t03_neuronio.mp   # compila e grava
.tac/.cpp/executável em output/
```

Fluxo interno:

```
fonte .mp
→ Lexer.initialize(source) + Parser.parse()
→ SemanticAnalyzer.analyze(program)
→ TACGenerator.generate(program, output_name)      # output/*.tac
→ CppCodeGenerator.generate(tac, output_name)      # output/*.cpp
→ DriverGCC.compilar(...)                          # output/<exe> + minipar_worker
→ DriverGCC.executar(...)  ou  TACExecutor.execute (# simulação/paralelo simples)
```

Erros de fase usam códigos **2 (léxica), 3 (sintática), 4 (semântica), 5 (geração), 6 (build), 7 (execução)**; uso/arquivo → **1**. Na web, `POST /api/executar` com `inputs` opcional, corte 30 s / 1 MiB.

**Rede:**

```
# localhost (dois terminais, servidor primeiro)
.\output\programa.exe --hosts .\minipar.hosts --como computador_2
.\output\programa.exe --hosts .\minipar.hosts --como computador_1

# fork remoto (um worker por máquina)
.\minipar_worker.exe --porta 5001
.\programa.exe --hosts .\minipar.hosts
```

Ver `docs/rede.md`, `programas_teste/t01_multiterminal/README.md` e `minipar.hosts` na raiz.

## 12.3 Observações técnicas e limitações

- **Simplicidade didática:** inferência simples com `unknown` compatível; `auto` no C++ gerado; sockets com `minipar.hosts` permitem localhost e 2 máquinas ajustando IP:porta e firewall.

- **Limitações conscientes (documentadas):**
  - `list.append`, `string.split`, `string.strip` são aceitos pelo semântico, mas **ainda sem lowering completo para TAC/C++**; programas novos (incl. T11) evitam listas e usam `string/int/float/bool/char`. `matrix(...)` mapeia para `__make_matrix` mas inferência de listas aninhadas é limitada (T10 usa regra base-3).
  - Canais só no topo; não é possível criar canal dentro de função/bloco (ver decisão 15).
  - Execução TAC simulada (intercalação + FIFO em memória) não representa paralelismo real nem rede.
  - `fork em host` exige `minipar_worker` na outra máquina; validação automatizada é localhost, 2 máquinas reais pendente (T41).
  - Hub de Conexão: o teste com dois computadores usou o Radmin como rede virtual e levou às correções de descoberta e de IP anunciado (§9.1); o procedimento validado ainda não está registrado em `docs/rede.md`. A descoberta é por broadcast UDP e pode falhar em redes que o bloqueiam — nesse caso o hub é adicionado manualmente pelo IP e porta; o firewall precisa liberar a porta TCP do hub (padrão 5010), a UDP 5011 e a porta do canal do programa. Convites de execução só existem para exemplos que têm par `_cliente`/`_servidor`; quem inicia usa o código do editor e quem aceita recebe o arquivo original do par.
  - O botão "Baixar .exe" entrega somente o executável do programa; o `minipar_worker` usado por `fork ... em host` não é incluído no download (use o CLI, que grava ambos em `output/`).
  - CPUlator no T11 manual (screenshot pendente); T8–T10 dependem de nº de CPUs — runner valida marcos, não tempos exatos.
- **Melhorias futuras sugeridas:** recuperação sintática mais rica e testes unitários por fase (já parcialmente implementados em 2026.2 — ver §5/§11), separar `int`/`float` estritamente, lowering pendente de builtins de lista/string, matcher de timeout configurable, e cobertura/total de testes verdes completos no CI.

---

## 13. Conclusão

O **minipar-improvement 2026.2** atende ao núcleo do enunciado 2026.2: analisadores léxico (com `NEWLINE/INDENT/DEDENT`), sintático (LL(1) + AST/Visitor) e semântico (2 passagens + `DiagnosticBag`), tabela de símbolos com escopos aninhados e herança, TAC de três endereços com executor e otimizador didático, transpilação para **C++17** com driver `g++` e runtime completo (canais TCP tipados, afinidade por núcleo, workers remotos), OO com herança, `PAR`/`FORK`/`JOIN`, laços `while`, `do … while` e `for`, e interface CLI + IDE web offline com download do executável, página de Ajuda e **Hub de Conexão** para executar programas cliente/servidor em dois computadores diretamente pelo editor. Os **11 testes T1–T11** passam com evidências versionadas, correções de backend registradas (uma pendente: tipo de `parse_ints`), e benchmarks com speedups medidos (T4 15,62×; T8–T10 ~3,1× contra meta >2,5×). Restam pendências de entrega bem mapeadas (GitHub Project com `read:project`, matriz de 17 itens, CPUlator/2 máquinas, clone limpo, PDF final/tag/release) — nenhuma inventada neste relatório.

---

## 14. Links úteis

1. Repositório 2026.2: https://github.com/johnUFAL/minipar-improvement-2026.2
2. Repositório 2026.1 (base): https://github.com/luizwhirl/minipar-improvement/tree/main
3. Especificações do projeto (professor): `Compil-AB2` [2026]
4. Wireframe da nova interface: https://www.figma.com/design/NXKjTBkAbP0foCPBm0ge0M/MiniPar-2026.2
5. Este relatório (draft Markdown 2026.2): `docs/relatorio/RELATORIO-2026.2.md` (PDF a gerar a partir dele)
6. Relatório de testes 2026.2: `docs/relatorio/RELATORIO-TESTES-2026.2.md` + `resultados/testes_2026_2/`
7. Gramática 2026.2: `docs/gramatica.md`; BNF legado 2026.1: `docs/uml/BNF-grammaMiniPAR2026-1.md`
8. Backlog: `docs/backlog.md`; plano por issue: `ANALISE-ISSUES-E-PLANO.md`
9. Rede/fork remoto: `docs/rede.md`, `minipar.hosts`
10. Pseudocódigos: `docs/relatorio/pseudocodigos.md`, `docs/pseudocodigo_tabela_simbolos.md`

---

## 15. Referências

**Baseadas nos materiais do professor:**


1. AHO, Alfred V.; LAM, Monica S.; SETHI, Ravi; ULLMAN, Jeffrey D. *Compiladores: princípios, técnicas e ferramentas*. 2. ed. São Paulo: Pearson Addison-Wesley, 2008.
2. SANTIAGO, Judson. *Compiladores: Introdução, Análise Semântica e Tabela de Símbolos*. Material de aula (Slides). [S.l.: s.n.], [20--].
3. UNIVERSIDADE FEDERAL DA PARAÍBA (UFPB). Departamento de Sistemas e Computação (DSC). *Parte 4: Tabela de Símbolos e Verificação Semântica de Tipos*. [S.l.: s.n.], [20--].
4. DOMINGUEZ, Arturo Hernandez. *Elementos de Implementação Utilizando Componentes*. Material de apoio da disciplina de Compiladores. Maceió: Universidade Federal de Alagoas, Instituto de Computação, [202-].
5. DOMINGUEZ, Arturo Hernandez. *Exemplo programa MiniParOO converter para código C*. Material de apoio da disciplina de Compiladores. Maceió: Universidade Federal de Alagoas, Instituto de Computação, [202-].
6. MASSACHUSETTS INSTITUTE OF TECHNOLOGY (MIT). *Computation Structures: Structs e Memória*. Material de aula. Cambridge: MIT OpenCourseWare, [20--].
7. UNIVERSIDADE FEDERAL DE ALAGOAS (UFAL). Instituto de Computação. *Apostila LFeC II: Introdução à Compilação*. Material de apoio. Maceió: UFAL, [20--].


**Adicionais (tecnologias e links):**


1. HAVERBEKE, Marijn. *CodeMirror: In-browser code editor*. 2026. Disponível em: https://codemirror.net/. Acesso em: 15 maio 2026.
2. ISO/IEC. *ISO/IEC 14882:2020: Programming Languages — C++*. Genebra: ISO, 2020. Disponível em: https://isocpp.org/std/the-standard. Acesso em: 28 maio 2026.
3. LOUDEN, Kenneth C. *Compiladores: princípios e práticas*. São Paulo: Pioneira Thomson Learning, 2004.
4. PALLETS PROJECTS. *Flask Documentation (3.0.x)*. 2023. Disponível em: https://flask.palletsprojects.com/. Acesso em: 30 abril 2026.
5. PYTHON SOFTWARE FOUNDATION. *The Python 3 Language Reference*. 2026. Disponível em: https://docs.python.org/3/reference/. Acesso em: 9 maio 2026.
6. PYTHON SOFTWARE FOUNDATION. *pytest documentation*. Disponível em: https://docs.pytest.org/. Acesso em: 2026.
7. ISO. *ISO/IEC 19501:2005 — UML 2.0*. Genebra: ISO, 2005. PlantUML diagrams em `docs/uml/*.puml`. Acesso em: 2026.


---

*Fim do relatório — draft Markdown 2026.2. Próximo passo sugerido: revisar, gerar o PDF com ReportLab a partir deste arquivo e publicar como release `v2026.2`.*